

**DOSSIER DE PROJET POUR PASSAGE DU TITRE
RNCP DEVELOPPEUR WEB ET WEB MOBILE**

**ANNEE 2025-2026 (CDPI LA
PLATEFORME/EUREKA)**

Josselin Fauconnier

Sommaire

I)Projet site perso (<https://github.com/Josselin-Fauconnier/site-perso>) P3-7

- a)Réalisation d'une interface utilisateur statique /SEO (CP3)
- b) Documenter le déploiement d'une application dynamique web ou web mobile (CP8)

II) Projet Bibliotech (<https://github.com/Josselin-Fauconnier/bibliotech>)P8-47

- A) Configuration de l'environnement de Travail (CP1)
- B)Réalisation d'une interface utilisateur statique (CP3)
- C)Développer la partie dynamique des interfaces utilisateur(CP4)
- D)Mettre en place une base de données relationnelle (CP5)
- E)Développer des composants d'accès aux données SQL et NoSQL (cp6)
- F)Développer des composants métier coté serveur (CP7)

III) Axe d'amélioration P48-49

I)SITE PERSO

Présentation du projet

Le portfolio a été développé dans le cadre de ma formation de développeur à La Plateforme, comme exercice pratique de front-end. L'objectif était de concevoir un site en HTML, CSS et JavaScript natif, sans framework, afin de me présenter, d'exposer mes compétences techniques et de mettre en avant ms projets les plus récents et/ou abouti.

Le site repose sur une architecture en JavaScript orienté objet avec des classes servant à la gestion du thème (ThemeManager) et de la langue (LanguageManager). Il propose un mode clair/sombre qui détecte automatiquement la préférence système de l'utilisateur via prefers-color-scheme, tout en conservant le choix manuel grâce à localStorage. Le site est également bilingue français/anglais, avec une bascule entre les deux versions. Une attention particulière a été portée à l'accessibilité, avec un lien d'évitement, des attributs aria-label, une navigation au clavier et une structure HTML5 sémantique, ainsi qu'au référencement, via la présence d'un sitemap.xml, d'un robots.txt et de meta descriptions adaptées.

Ce projet m'a ainsi permit de développer ma capacité a structurer un site web monopage , en respectant les bonnes pratiques de code, et d'accessibilité.



a) Réalisation d'une interface utilisateur statique /SEO (CP3)

Pour ce projet, j'ai cherché à faire un référencement naturel basique. Chaque page dispose d'une balise meta description unique et adaptée à sa langue, résumant qui je suis, et la balise lang du document HTML est définie sur FR ou en EN selon la version consultée. Le site étant bilingue, le multilinguisme est géré via les balises link rel="alternate" hreflang présentes dans le head de chaque page, en indiquant systématiquement les trois variantes possibles : français, anglais et x-default, qui sert de version par défaut quand la langue du visiteur ne correspond à aucune des deux. Chaque page possède également une balise canonical pointant vers sa propre URL, ce qui évite que Google considère les deux versions comme un contenu dupliqué et précise explicitement quelle URL doit être indexée pour chaque langue.

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="description" content="Porfolio de Josselin Fauconnier.Developpeur junior en formation">
  <title>Portfolio de Josselin Fauconnier</title>
  <meta name="google-site-verification" content="D1kxhDJHa_PzBCedhtSoimTe4yJ8VvJnab_bUFdapzM" />
  <link rel="canonical" href="https://josselin-fauconnier.students-laplateforme.io/">
  <link rel="alternate" hreflang="fr" href="https://josselin-fauconnier.students-laplateforme.io/">
  <link rel="alternate" hreflang="en" href="https://josselin-fauconnier.students-laplateforme.io/en/">
  ...<link rel="alternate" hreflang="x-default" href="https://josselin-fauconnier.students-laplateforme.io/">
  ...<link rel="stylesheet" href="style.css">
</head>
```

L'exploration du site par les robots est encadrée par deux fichiers complémentaires. Le robots.txt, premier point consulté par un robot d'indexation, contient deux directives volontairement simples : User-agent: *, qui s'applique à l'ensemble des robots sans distinction, suivi de Allow: /, qui autorise explicitement l'exploration de toutes les pages, sans restriction . Ce choix se justifie par la nature du projet : un portfolio statique entièrement public. La dernière ligne, Sitemap: <https://josselin-fauconnier.students-laplateforme.io/sitemap.xml>, indique directement aux robots où trouver le plan du site, ce qui améliore la vitesse de prise en compte des pages lors du crawl.

```
User-agent: *  
Allow: /  
  
Sitemap: https://josselin-fauconnier.students-laplateforme.io/sitemap.xml
```

Le sitemap.xml utilise le protocole Sitemaps et liste les deux versions linguistiques de la page d'accueil, la version française (/) et la version anglaise (/en/). Chaque <url> comporte une balise <loc> indiquant l'adresse de la page et une balise <lastmod> datée de la dernière mise à jour du contenu, ce qui permet aux robots de prioriser le crawl des pages récemment modifiées. La relation entre les langues est portée uniquement par les balises hreflang du head HTML, ce qui évite une duplication inutile en ne le centrant que sur une tâche qui est celle de lister les divers URLs à indexer et leur mis-à-jours

```
<?xml version="1.0" encoding="UTF-8"?>  
<urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9">  
  <url>  
    <loc>https://josselin-fauconnier.students-laplateforme.io/</loc>  
    <lastmod>2026-06-21</lastmod>  
  </url>  
  <url>  
    <loc>https://josselin-fauconnier.students-laplateforme.io/en/</loc>  
    <lastmod>2026-06-21</lastmod>  
  </url>  
</urlset>
```

Enfin, j'ai enregistré le portfolio sur Google Search Console, via une balise meta name="google-site-verification" insérée dans le head de la page. Cette vérification permet de soumettre la sitemap directement à Google, de suivre l'indexation réelle des pages, de surveiller les requêtes de recherche menant vers mon site afin d'affiner le SEO en continue.

Dernière lecture

22/06/2026

Pages
découvertes

2

Vidéos
découvertes

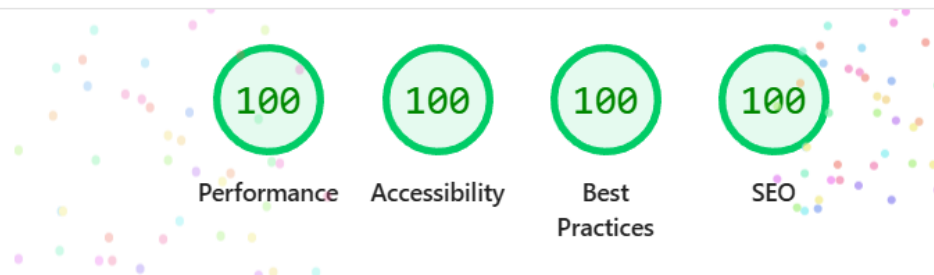
0



VOIR L'INDEXATION DES PAGES

VOIR L'INDEXATION DES PAGES
VIDÉO

Traitement du sitemap réussi

<https://josselin-fauconnier.students-laplateforme.io/>

b) Déploiement

Pour mettre le site en ligne, j'ai utilisé WinSCP, un logiciel qui permet d'envoyer des fichiers depuis un ordinateur vers un hébergeur (Plesk dans ce cas) Plesk. J'ai choisi de le faire en FTP.

Pour le faire, je me connecte avec WinSCP, puis j'ai envoyé les fichiers dans le dossier httpdocs, qui est le dossier que Plesk affiche publiquement sur internet. Je n'ai envoyé que les fichiers en lien direct avec le site (les pages HTML, le CSS, les scripts JS, le sitemap et le robots.txt).

C:\Users\fauco\Desktop\laplateforme\site perso\site-perso\				/httpdocs/				
Nom	Taille	Type	Date de modification	Nom	Taille	Date de modification	Droits	Propriét...
..		Répertoire parent	22/06/2026 13:47:51	autocompletion		10/11/2025 10:15:41	rwxf--f--	josselin--
en		Dossier de fichiers	21/06/2026 15:47:54	bigjob		02/11/2025 22:46:14	rwxf--f--	josselin--
JS		Dossier de fichiers	21/06/2026 15:42:05	en		22/06/2026 10:14:25	rwxf--f--	josselin--
.gitignore	1 KB	Fichier source Git I...	19/06/2026 09:57:08	JS		22/06/2026 13:40:37	rwxf--f--	josselin--
index.html	6 KB	Brave HTML Docu...	22/06/2026 13:47:51	livre d'or		30/09/2025 15:39:41	rwxf--f--	josselin--
robots.txt	1 KB	Document texte	21/06/2026 15:50:11	memory		07/11/2025 09:48:12	rwxf--f--	josselin--
sitemap.xml	1 KB	Microsoft Edge HT...	22/06/2026 14:50:05	memory2		21/10/2025 21:53:53	rwxf--f--	josselin--
style.css	8 KB	Fichier source CSS	19/06/2026 09:57:08	module connexion		07/08/2025 14:22:07	rwxf--f--	josselin--
				oclock		31/10/2025 10:19:56	rwxf--f--	josselin--
				index.html	6 KB	22/06/2026 13:48:29	rw-r--r--	josselin--
				robots.txt	1 KB	21/06/2026 15:50:11	rw-r--r--	josselin--
				sitemap.xml	1 KB	22/06/2026 14:52:10	rw-r--r--	josselin--
				style.css	8 KB	19/06/2026 09:57:08	rw-r--r--	josselin--

Une fois les fichiers envoyés, j'ai vérifié que le site fonctionne bien en vérifiant que les boutons (changement de langue, changement de thème ,skip-link) fonctionnent.

II)Bibliotech

Présentation du projet

BiblioTech est un projet personnel développé en autonomie en cours d'année, en m'inspirant du projet Cinetech (<https://github.com/Josselin-Fauconnier/Cinetech>) réalisé en formation à La Plateforme autour de la gestion d'une bibliothèque de films favoris . J'ai dans l'idée repris cette logique applicative d'affichage , de commentaire et des listes de favoris en y rajoutant celles d'authentification et de modération en l'appliquant au monde du livre, un domaine qui m'intéresse . L'objectif était de concevoir une application de gestion de bibliothèque personnelle, permettant de rechercher des livres, de les organiser en listes de lecture et d'échanger autour de ses lectures via un système de commentaires.

Le site repose sur une architecture front et back , avec un frontend en HTML/CSS/JS natif (mobile first, Flexbox/Grid,) et un backend Node.js/Express 5 connecté à une base de données MySQL. La recherche et la consultation des livres s'appuient sur l'API publique Open Library, tandis qu'un système d'authentification JWT couplé à bcrypt sécurise les comptes utilisateurs et personnalise l'expérience (listes de lecture privées, commentaires, profil). Un tableau de bord administrateur permet à l'administrateur de gérer les utilisateurs et de modérer les commentaires.

Les schémas de validation Zod, centralisés dans un module partagé du projet et consommés par le serveur, garantissent la cohérence des données reçues, tandis que des tests unitaires (Vitest) couvrent ces schémas ainsi que le middleware d'authentification.

Le site respecte les bonnes pratiques d'accessibilité web : structure HTML5 sémantique (header, nav, main, footer), lien d'évitement vers le contenu principal, labels associés à chaque champ de formulaire, et attributs ARIA (aria-live, aria-label, role="dialog", aria-describedby) pour les messages dynamiques, les modales et les zones de recherche.

La protection des données personnelles a été fait pour correspondre aux attentes du RGPD ;les mots de passe sont hashés avec bcrypt, les tokens JWT sont limités dans le temps et stockés en cookie httpOnly, et la suppression d'un compte entraîne une anonymisation des données (email, nom d'utilisateur, mot de passe) plutôt qu'un simple effacement.

Ce projet a eu pour objectif de mettre en pratiques la plus part des concepts que j'ai vue lors de la formation en étant entièrement autonome en lien avec un domaine qui m'intéresse.

a) Configuration de l'environnement de Travail (CP1)

Avant de coder, j'ai mis en place mon environnement de travail sur Visual Studio Code, avec Node.js comme environnement d'exécution et npm pour la gestion des dépendances du projet.

```
{
  "name": "bibliosauv",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "type": "module",
  "scripts": {
    "dev": "node --watch src/backend/server.js",
    "test": "vitest",
    "test:coverage": "vitest run --coverage"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "dependencies": {
    "bcrypt": "^6.0.0",
    "cors": "^2.8.6",
    "dotenv": "^17.4.2",
    "express": "^5.2.1",
    "jsonwebtoken": "^9.0.3",
    "mysql2": "^3.22.2",
    "zod": "^4.3.6"
  },
  "devDependencies": {
    "@vitest/coverage-v8": "^4.1.5",
    "vitest": "^4.1.5"
  }
}
```

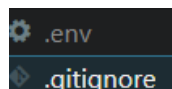
Le fichier package.json sert à configurer la configuration et définit trois scripts qui sont :

- `npm run dev` , qui lance le serveur avec l'option `--watch` de Node.js permettant un redémarrage automatique à chaque modification du code.

- `npm test` ,qui exécute la suite de tests unitaires avec Vitest.
- `npm run test:coverage` qui génère un rapport de couverture de tests via `@vitest/coverage-v8`

Les dépendances de production sont là pour les besoins fonctionnels et sécuritaires du projet : `express` pour le framework serveur, `mysql2` pour l'accès à la base de données, `bcrypt` pour le hachage des mots de passe, `jsonwebtoken` pour l'authentification par jeton, `zod` pour la validation des données, `cors` pour la gestion des origines autorisées, et `dotenv` pour la gestion de la configuration.

Le projet est versionné avec Git et je l'héberge sur un dépôt distant GitHub, ce qui me permet de conserver l'historique de mes modifications, de revenir à une version antérieure en cas de besoin. Les informations sensibles tel que les identifiants de connexion à la base de données, clé secrète de signature des jetons JWT, port d'écoute du serveur, origine CORS autorisée qui sont dans un fichier `.env` chargé au démarrage de l'application via la bibliothèque `dotenv` . Le fichier n'est pas versionné (mis en `.gitignore`) dans le dépôt Git afin d'éviter toute fuite de secret risquant de compromettre la sécurité du code.



Pour la base de données, j'ai utilisé `phpMyAdmin` comme interface d'administration pour créer les tables et gérer leur structure et leurs contraintes. J'ai exporté depuis `phpMyAdmin` un script SQL de création de la base de données, que j'ai conservé dans le dossier `DB` du projet. La connexion à la base s'effectue via un pool de connexions (`mysql2/promise`, `mysql.createPool`) afin de pouvoir traiter plusieurs requêtes simultanées sans dégrader les performances de l'application.

Pour fiabiliser le code, j'ai mis en place des tests unitaires ,avec Vitest, couvrant les schémas de validation `Zod` et le middleware d'authentification `isAuthenticated`, ce qui me permet de sécuriser mes évolutions de code sans craindre de régression sur les fonctionnalités déjà validées.

b) Réalisation d'une interface utilisateur statique (CP2)

Chacune des neuf pages HTML de BiblioTech ont été structurées autour des balises sémantiques HTML5 header, nav, main et footer afin de donner un sens structurel au contenu et de faciliter la navigation pour les technologies d'assistance. Chaque page comporte un lien d'évitement vers le contenu principal (Aller au contenu principal), placé en tout premier élément du corps de page, ce qui permet à un utilisateur naviguant au clavier ou avec un lecteur d'écran de passer directement au contenu sans devoir parcourir la navigation. Les champs de formulaire sont systématiquement associés à un label, y compris lorsque celui-ci est masqué visuellement par la classe sr-only pour ne rester accessible qu'aux lecteurs d'écran, et les zones de recherche utilisent l'attribut role="search". Les messages d'état dynamiques, comme sur la page de recherche de livres, sont annoncés aux lecteurs d'écran grâce à l'attribut aria-live="polite", sans interrompre la lecture en cours.

```

<a href="#main-content" class="skip-links">Aller au contenu principal</a>

<header class="header">
| <a href="/" class="site-name">BiblioTech</a>
</header>
<nav id="main-nav" class="nav"></nav>

<main id="main-content" class="main">
|
| <section class="hero">
| | <h1 class="hero__title">Votre bibliothèque en ligne</h1>
| | <p class="hero__subtitle">Découvrez des millions de livres et organisez vos lectures.</p>
|
| | <form id="search-form" class="search-form hero__search" role="search">
| | | <label for="search-input" class="sr-only">Rechercher un livre</label>
| | | <input
| | | | type="search"
| | | | id="search-input"
| | | | class="search-form__input"
| | | | placeholder="Titre, auteur, ISBN..."
| | | | autocomplete="off"
| | | >
| | | <button type="submit" class="search-form__btn">Rechercher</button>
| | </form>
| </section>
|
| <section class="trending">
| | <h2 class="section-title">Livres du moment</h2>
| | <p id="status-message" class="status" aria-live="polite"></p>
| | <ul id="books-grid" class="books-grid"></ul>
| </section>
|
</main>

<footer id="main-footer" class="footer"></footer>

<script type="module" src="/js/pages/index.js"></script>

```

Pour la mise en forme, j'ai décidé d'avoir une approche mobile first avec Flexbox et Grid, comme indiqué dans le fichier style.css partagé par l'ensemble des pages. Les points de rupture sont définis à 768px et 1200px, ce qui permet d'adapter progressivement la densité d'affichage du plus petit écran mobile jusqu'à l'écran de bureau. Il ya aussi un système de changement de luminosité prise en compte via la media query prefers-color-scheme, en redéfinissant les variables CSS de couleur plutôt qu'en dupliquant les règles de style.

```

@media (prefers-color-scheme: dark) {
  :root:not([data-theme="light"]) {
    --color-primary: #4a7fcb;
    --color-primary-light: #5a8fd5;
    --color-nav: #7c3aed;
    --color-accent: #f59e3a;
    --color-bg: #0f1117;
    --color-surface: #1a1d2e;
    --color-text: #e2e8f0;
    --color-text-muted: #a0aec0;
    --color-border: #2d3748;
    --color-error: #fc8181;
    --shadow: 0 5px 10px rgba(0, 0, 0, 0.4);
    --shadow-hover: 0 10px 25px rgba(0, 0, 0, 0.6);
  }
}

```

```

@media (min-width: 768px) {
  .main {
    padding: var(--space-section);
  }

  .nav {
    justify-content: center;
    gap: var(--space-component);
    height: 44px;
  }

  .nav__burger {
    display: none;
  }

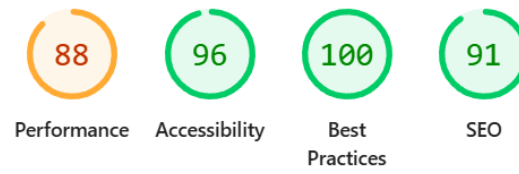
  .nav__menu {
    display: flex;
    flex-direction: row;
    align-items: center;
    width: auto;
    padding: 0;
    gap: var(--space-component);
  }
}

@media (min-width: 1200px) {
  .main {
    padding: var(--space-layout);
  }
}

```

L'application n'étant pas

déployée n'étant pas déployé je n'ai pas forcément mis l'accent sur tout ce qui est nécessaire pour le SEO le plus adéquate ce qui n'empêche pas d'avoir un bon score selon lighthouse en local .



C)Développer la partie dynamique des interfaces utilisateur(CP4)

Cela est divisé en trois dossiers aux responsabilités distinctes. Le dossier api regroupe les fonctions d'appel au serveur `searchBooks`, `getTrendingBooks`, `getWorkDetails`, `getAuthorName` dans `backend.js` chacune est responsable d'interroger un endpoint précis et de vérifier la réponse avant de la renvoyer. Le dossier pages contient un fichier par écran, qui orchestre l'affichage et les interactions propres à cette page sans jamais détailler comment une donnée est récupérée. Le dossier utils réunit les fonctions transverses réutilisées par plusieurs pages : `apiFetch` pour les appels nécessitant une session, `navBarre` pour la navigation commune, `pagination` pour les listes de résultats paginées et `bookCard` pour .

1)Dossier API

Le fichier `backend.js` sert d'interface entre les pages et l'API du serveur pour tout ce qui concerne les livres ; il contient quatre fonctions, chacune correspondant à un besoin différent.

`getTrendingBooks` interroge l'endpoint `/api/books/trending` sans paramètre : contrairement à une recherche, aucun critère n'est à transmettre, puisque c'est l'API Open Library elle-même qui détermine la sélection de livres tendance du moment qui est une sélection qui peut évoluer au jour le jour côté Open Library, sans que l'appel ait besoin de le refléter par un paramètre. Une fois la réponse reçue, la fonction vérifie la propriété `ok` avant de la transformer, puis reconstruit un objet `{ docs, numFound }` à partir du tableau `works` renvoyé par Open Library, en utilisant l'opérateur nulle (`??`) pour retomber sur un tableau vide si la propriété est absente. Cette reconstruction permet à `books.js` de traiter indifféremment les livres tendance et les résultats d'une recherche avec exactement la même fonction `renderBooks`, sans avoir à connaître la différence de format entre les deux réponses brutes de l'API

```
export async function getTrendingBooks() {
  const response = await fetch('/api/books/trending');

  if (!response.ok) {
    throw new Error(`Erreur serveur : ${response.status}`);
  }

  const data = await response.json();
  return { docs: data.works ?? [], numFound: data.works?.length ?? 0 };
}
```

`searchBooks` prend en paramètres la requête de l'utilisateur et la page courante (avec une valeur par défaut de 1 si elle n'est pas précisée), et construit l'URL d'appel en passant la requête dans `encodeURIComponent` avant de l'insérer dans le paramètre `q` : cet encodage transforme les caractères spéciaux (espaces, accents, `&`, `?`) en une forme sûre pour une

URL, ce qui évite qu'une recherche contenant ce type de caractère ne casse la requête ou ne soit mal interprétée par le serveur. Contrairement à `getTrendingBooks`, cette fonction renvoie directement le résultat de `response.json()` sans le retransformer, puisque le format renvoyé par l'endpoint de recherche correspond déjà à ce qu'attend `books.js` (un tableau `docs` et un total `numFound`).

```
export async function searchBooks(query, page = 1) {
  const response = await fetch(`/api/books?q=${encodeURIComponent(query)}&page=${page}`);

  if (!response.ok) {
    throw new Error(`Erreur serveur : ${response.status}`);
  }

  return response.json();
}
```

`getWorkDetails` et `getAuthorName` suivent le même principe d'appel-puis-vérification, mais avec deux comportements d'erreur différents, choisis selon l'importance de la donnée pour l'affichage. `getWorkDetails` lève une erreur si le livre n'est pas trouvé, car sans ces données il n'y a rien à afficher sur la page de détail, l'erreur est alors interceptée plus haut, dans `detailBook.js`, pour afficher un message d'indisponibilité. `getAuthorName`, à l'inverse, ne lève pas d'erreur si l'auteur n'est pas trouvé : elle renvoie simplement le texte de repli "Auteur introuvable", car l'absence du nom d'un auteur ne doit pas empêcher l'affichage du reste de la fiche du livre. Cette fonction commence aussi par retirer le préfixe `/authors/` de l'identifiant fourni par Open Library (`key.replace('/authors/', '')`), car l'API renvoie cet identifiant sous une forme complète qu'il faut nettoyer avant de l'insérer dans l'URL de la requête suivante.

```
export async function getWorkDetails(id) {
  const res = await fetch(`/api/books/work/${id}`);
  if (!res.ok) throw new Error('Le livre est introuvable.');
  return res.json();
}

export async function getAuthorName(key) {
  const id = key.replace('/authors/', '');
  const res = await fetch(`/api/books/author/${id}`);
  if (!res.ok) return 'Auteur introuvable';
  const data = await res.json();
  return data.name ?? "Le nom n'est pas renseigné";
}
```


2) Dossier Page

a) Admin.js

La page admin.js commence par une redirection si le rôle stocké en localStorage n'est pas admin ,c'est un filtre pour améliorer l'UX , l'accès réel aux routes /api/admin/* étant revérifié côté serveur par le middleware isAdmin.

La page gère quatre sections de données indépendantes : utilisateurs, commentaires, comptes bannis temporairement, comptes bannis définitivement , chacune avec sa propre variable de page courante (usersPage, commentsPage, etc.). Les quatre fonctions de chargement (loadUsers, loadComments, loadBannedTemp, loadBannedPerm) suivent exactement le même schéma : afficher un message de chargement, vider le tableau, appeler apiFetch avec le numéro de page courant, construire les lignes du tableau, puis appeler renderPagination avec un callback qui met à jour uniquement la page de cette section avant de la recharger .

Le bannissement et le débannissement passent par une boîte de dialogue de confirmation . La fonction getFocusable liste tous les éléments interactifs présents dans la modale (boutons, champs, liens), et openModal s'en sert pour positionner le focus sur le premier d'entre eux à l'ouverture, puis pour intercepter les touches Tab et Maj+Tab et boucler le focus à l'intérieur de la modale plutôt que de le laisser s'échapper vers le reste de la page. La touche Échap déclenche la fermeture, et closeModal restitue le focus à l'élément qui avait ouvert la modale (le bouton "Bannir" ou "Débannir" correspondant à la ligne concernée). Cette même mécanique de modale est réutilisée à l'identique pour les deux confirmations, seul leur contenu (champs de raison et de durée pour le bannissement, simple confirmation pour le débannissement) diffère.

Une fois la confirmation validée, la durée de bannissement saisie est convertie en nombre ou laissée à null si le champ est vide, ce qui correspond à un bannissement permanent côté serveur. Comme une seule action (bannir ou débannir un compte) affecte simultanément plusieurs vues du tableau de bord, les trois fonctions de chargement concernées (loadUsers, loadBannedTemp, loadBannedPerm) sont systématiquement rappelées ensemble après chaque confirmation, afin que les quatre tableaux restent cohérents entre eux sans nécessiter de rechargement complet de la page. Le temps restant avant l'expiration d'un bannissement temporaire est calculé à l'affichage par la fonction

timeRemaining, à partir de la différence entre la date actuelle et banned_until , la valeur n'est donc jamais stockée, mais recalculée à chaque rendu du tableau pour rester exacte.

b)Books.js

Le script gère l'état de la recherche dans deux variables de module, currentQuery et currentPage.

Quand le formulaire de recherche est soumis, la page revient toujours à 1 avant de lancer la nouvelle recherche. Sans cette réinitialisation, une recherche lancée depuis la page 5 d'un résultat précédent resterait bloquée sur cette page 5, alors qu'elle devrait recommencer au début.

Au chargement de la page, le script vérifie d'abord si un paramètre q est présent dans l'URL. C'est le cas après une redirection depuis le champ de recherche de la page d'accueil : la recherche correspondante est alors lancée directement, avec les bons résultats déjà affichés. Si aucun paramètre n'est présent, la fonction loadTrending prend le relais et affiche les livres tendance du moment, pour que la page ne soit jamais vide à l'arrivée.

La fonction loadBooks commence par vider la grille de résultats et la zone de pagination avant de lancer le nouvel appel, ce qui évite qu'un ancien résultat reste affiché pendant le chargement du suivant. Une fois la réponse de searchBooks reçue, le nombre total de pages est calculé à partir de numFound divisé par 20 (la taille de page fixée par l'API), et un message contextuel précise le nombre de résultats trouvés ainsi que la position dans la pagination. Le callback transmis à renderPagination déclenche un défilement fluide vers le haut de la page avant de recharger les résultats de la nouvelle page, pour que l'utilisateur ne reste pas visuellement en bas d'une liste qui vient de changer.

La construction de chaque carte livre passe par createBookCard, importée depuis bookCard.js . Les différents états de la requête (chargement, résultat vide, erreur) sont centralisés dans la fonction setStatus, qui met à jour le message affiché et bascule une classe CSS d'erreur selon le cas, plutôt que de dupliquer cette logique à chaque appel.

c)detailBook.js

La page commence par récupérer l'identifiant du livre dans l'URL. Si cet identifiant est absent, le script redirige immédiatement vers la page de recherche : sans lui, aucune des fonctions suivantes ne peut fonctionner. La fonction loadBook récupère ensuite les détails du livre via getWorkDetails. Chaque information est affichée seulement si elle existe : l'auteur est recherché séparément via getAuthorName, l'année et la description ne s'affichent que si l'API les fournit, et la couverture bascule vers un texte de remplacement si aucune image n'est disponible. Cette vérification systématique évite d'afficher des

champs vides ou la mention "undefined" pour les livres dont la fiche Open Library est incomplète.

La fonction `loadComments` affiche la liste des commentaires existants, avec l'auteur, le contenu et la date de chacun. Les boutons "Éditer" et "Supprimer" ne sont ajoutés que sur les commentaires dont l'auteur correspond au nom d'utilisateur stocké en local l'autorisation réelle étant vérifiée côté serveur. La suppression retire directement la ligne du DOM après confirmation du serveur. L'édition remplace le texte du commentaire par une zone de saisie déjà remplie avec le contenu actuel ; à l'enregistrement, le serveur est notifié, puis le texte affiché et la zone de saisie sont mis à jour sans recharger la liste entière.

La fonction `loadLists` alimente le menu déroulant permettant d'ajouter le livre à une liste de lecture, en listant les listes existantes de l'utilisateur, ou en affichant un message s'il n'en a encore aucune. Le formulaire d'ajout envoie l'identifiant de la liste choisie et l'identifiant du livre au serveur, puis affiche un message de réussite ou d'échec.

Le formulaire de commentaire suit le même schéma de gestion d'erreur que `register.js` et `lists.js` : en cas de refus du serveur, le premier message d'erreur structuré par Zod est extrait et affiché. En cas de succès, le champ de saisie est vidé et la liste des commentaires est rechargée pour afficher immédiatement le nouveau commentaire. Pour finir, le formulaire de commentaire et la section d'ajout à une liste ne s'affichent que si un nom d'utilisateur est présent en local ,ce qui fait qu'un visiteur non connecté peut consulter la fiche du livre, mais pas interagir avec elle.

d) `index.js`

Le formulaire de recherche de la page d'accueil ne lance pas de recherche directement. Il redirige vers `/html/books.html` en transmettant la requête saisie comme paramètre d'URL. La page d'accueil est juste un point d'entrée , qui se contente d'afficher les livres tendance au chargement, tandis que `books.html` prend en charge l'affichage complet des résultats et la pagination.

La fonction `loadTrending` récupère les livres tendance via `getTrendingBooks` et les affiche, ou montre un message d'erreur si l'appel échoue. Chaque carte livre est construite par `createBookCard`, importée depuis `bookCard.js` .La fonction `setStatus` sert à centralisé l'affichage des différents messages d'état (chargement, vide, erreur) pour éviter de répéter cette logique à chaque cas.

e) `listdetail.js`

Le code récupère l'identifiant de la liste dans l'URL et redirige vers `/html/lists.html` s'il est absent. La fonction `loadListName` récupère ensuite l'ensemble des listes de l'utilisateur via `apiFetch`, puis cherche celle dont l'identifiant correspond pour en afficher le nom. Il n'existe pas d'endpoint dédié pour récupérer une seule liste par son identifiant : le script récupère donc toutes les listes et filtre côté client pour trouver la bonne.

La fonction `loadListBooks` récupère les livres associés à cette liste. Comme la base ne stocke que l'identifiant de chaque livre (`book_id`) et pas ses détails, une fonction `createBookCard` locale à ce fichier va chercher les informations complètes de chaque livre via `getWorkDetails` avant de construire la carte correspondante. Si un livre n'est plus accessible auprès d'Open Library, cette fonction renvoie `null` plutôt que de faire échouer tout l'affichage, et la carte correspondante est simplement ignorée.

Le bouton "Retirer" de chaque carte supprime le livre de la liste côté serveur, puis retire la carte du DOM.

f)lists.js

La page commence par une redirection si aucun nom d'utilisateur n'est présent en `localStorage`. Ce contrôle est avant tout pour l'UX. L'accès réel à `/api/lists` est filtré côté serveur par le middleware `isAuthenticated`. La fonction `loadLists` récupère ensuite les listes de l'utilisateur via `apiFetch`. Chaque liste est affichée dans une carte avec son nom. Si l'utilisateur n'a encore créé aucune liste, un message l'indique à la place.

La création d'une liste suit le même schéma de gestion d'erreur que `register.js`. Si le serveur refuse la demande, le premier message d'erreur structuré par `Zod` est extrait et affiché. En cas de succès, le champ de saisie est vidé, la liste des cartes est rechargée, et le focus est déplacé sur le lien de la première carte affichée.

La suppression d'une liste repose sur un seul écouteur de clic posé sur le conteneur des listes, plutôt que sur un écouteur par carte. Le clic est identifié grâce à `e.target.closest('.list-card__delete')`. Une fois la suppression confirmée par le serveur, la carte est retirée du DOM et le focus est déplacé sur la carte suivante, la précédente, ou le champ de création de liste.

g)login.js

Le formulaire vérifie d'abord que les éléments du DOM existent avant de les utiliser, sinon une erreur explicite est levée. À la soumission, le bouton est désactivé pendant la requête pour éviter un double envoi. L'email est nettoyé des espaces superflus (`trim`), mais pas le mot de passe, car un espace peut en faire partie.

La requête est envoyée au serveur via `fetch`. Si le serveur refuse la connexion, le message d'erreur renvoyé est affiché tel que `identifiants invalides`, `compte banni`, etc. Si la connexion réussit, le rôle et le nom d'utilisateur sont stockés en local, puis l'utilisateur est redirigé vers le tableau de bord administrateur ou vers la recherche de livres selon son rôle. Si la requête échoue techniquement, un message générique l'indique. Le bouton est réactivé dans tous les cas à la fin.

h)profile.js

La page redirige vers la connexion si l'utilisateur n'est pas connecté. La fonction `loadProfile` récupère en un seul appel les informations du compte et ses commentaires paginés, puis affiche le nom d'utilisateur, l'email et la date d'inscription formatée en français.

Le formulaire de changement de mot de passe envoie l'ancien et le nouveau mot de passe au serveur, puis affiche directement le message renvoyé, en vert ou en rouge selon le résultat. Le formulaire est réinitialisé en cas de succès.

La suppression du compte demande d'abord une confirmation native du navigateur, rappelant que l'action est irréversible. Si le serveur confirme la suppression, les données locales sont effacées et l'utilisateur est redirigé vers la page de connexion ; sinon, un message d'erreur est affiché.

i)register.js

Le formulaire vérifie d'abord que les éléments du DOM existent, sinon une erreur explicite est levée. À la soumission, le bouton est désactivé pendant la requête. Le nom d'utilisateur et l'email sont nettoyés des espaces superflus, mais pas le mot de passe.

La requête est envoyée au serveur. Si l'inscription est refusée, le premier message d'erreur structuré par Zod est extrait et affiché — par exemple si le mot de passe ne respecte pas la politique de sécurité. Si l'inscription réussit, l'utilisateur est redirigé vers la page de connexion. En cas d'échec technique, un message générique l'indique. Le bouton est réactivé dans tous les cas à la fin.

3) Dossier Utlis

a)apiFetch.js

La fonction apiFetch centralise les appels au serveur nécessitant une session. Chaque requête inclut automatiquement les cookies via credentials: 'include', ce qui transmet le jeton JWT stocké en cookie httpOnly.

Si le serveur répond 401, la fonction déclenche immédiatement une déconnexion : le localStorage est vidé et l'utilisateur est redirigé vers la page de connexion. Sinon, un minuteur de session de trente minutes est relancé à chaque appel réussi, aligné sur la durée de vie du jeton côté serveur. Ce minuteur déconnecte automatiquement l'utilisateur côté client dès qu'il expire (30 minutes d'inactivité).

```

const SESSION_DURATION = 1000 * 60 * 30;

let logoutTimer = null;

function forceLogout() {
  clearTimeout(logoutTimer);
  localStorage.removeItem('username');
  localStorage.removeItem('role');
  window.location.href = '/html/login.html';
}

export function startSessionTimer() {
  if (!localStorage.getItem('username')) return;
  clearTimeout(logoutTimer);
  logoutTimer = setTimeout(forceLogout, SESSION_DURATION);
}

export async function apiFetch(url, options = {}) {
  const res = await fetch(url, { ...options, credentials: 'include' });

  if (res.status === 401) {
    forceLogout();
    throw new Error('Session expirée');
  }

  startSessionTimer();
  return res;
}

```

b)bookCard.js

La fonction createBookCard construit une carte livre avec createElement. Le titre et l'auteur sont insérés via.textContent, ce qui empêche tout contenu provenant d'Open Library de s'exécuter comme du HTML actif.

Plusieurs valeurs absentes sont gérées par des valeurs de repli : un auteur manquant affiche "L'auteur est inconnu", une année manquante n'affiche simplement rien, et l'identifiant du livre est nettoyé du préfixe /works/ renvoyé par l'API, comme pour les identifiants d'auteur dans backend.js. Si aucune couverture n'est disponible, un texte de remplacement s'affiche à la place de l'image, qui elle-même est chargée en différé (loading="lazy").

c)navBarre.js

Le thème clair/sombre est géré par trois fonctions : `getTheme` lit la préférence stockée en `localStorage` (ou "auto" par défaut), `applyTheme` pose ou retire l'attribut `data-theme` sur la page, et `toggleTheme` bascule entre les deux en tenant compte aussi de la préférence système (`prefers-color-scheme`) si aucun choix explicite n'a été fait. `updateToggleIcon` synchronise l'icône et le libellé accessible du bouton avec le thème actif.

La fonction `initNav` construit le menu différemment selon l'état de connexion : un simple lien vers la connexion si personne n'est identifié, ou un menu complet avec nom d'utilisateur, liens de navigation et bouton de déconnexion sinon — le lien vers le tableau de bord n'apparaissant que pour le rôle admin, un affichage conditionnel cosmétique comme expliqué précédemment. Le menu mobile (bouton burger) gère son état ouvert/fermé via `aria-expanded`, se referme automatiquement après le clic sur un lien, et la page actuelle est signalée par `aria-current` plutôt que par un simple style.

Le bouton de déconnexion appelle d'abord `/api/auth/logout`, puis vide le `localStorage` et redirige vers la connexion. La fonction `initFooter` ajoute un lien externe vers Open Library avec l'attribut `rel="noopener"`, qui empêche la page ouverte dans le nouvel onglet d'accéder à la fenêtre d'origine — une précaution de sécurité standard pour les liens `target="_blank"`.

d)pagination.js

La fonction `renderPagination` ne s'affiche pas si une seule page suffit. Elle construit ensuite un bloc nav avec un `aria-label` dédié, contenant un bouton précédent, les numéros de page, et un bouton suivant.

`getPageNumbers` limite l'affichage à la première page, la dernière, et les pages proches de la page courante, en insérant des points de suspension pour les pages masquées. Cela évite d'afficher des dizaines de boutons quand une recherche renvoie de nombreux résultats.

La page courante est signalée par `aria-current`, et les icônes des boutons précédent/suivant sont masquées aux lecteurs d'écran (`aria-hidden`) au profit d'un texte accessible équivalent.

d)Mettre en place une base de données relationnelle (CP5)

La base de données repose sur six tables. `users` stocke les comptes (identifiant, nom d'utilisateur, email, mot de passe haché, rôle). Autour d'elle s'organisent `reading_lists` et `list_books` pour les listes de lecture et les livres qu'elles contiennent, `comments` pour les avis laissés sur un livre, et deux tables de traçabilité, `login_attempts` et `user_events`, qui journalisent respectivement les tentatives de connexion échouées et les événements liés à un compte (inscription, connexion, suppression).

Table	Action	Lignes	Type	Interclassement	Taille	Perte
☐ comments	★ Parcourir Structure Rechercher Insérer Vider Supprimer	4	InnoDB	utf8mb4_0900_ai_ci	32,0 kio	-
☐ list_books	★ Parcourir Structure Rechercher Insérer Vider Supprimer	4	InnoDB	utf8mb4_0900_ai_ci	32,0 kio	-
☐ login_attempts	★ Parcourir Structure Rechercher Insérer Vider Supprimer	22	InnoDB	utf8mb4_0900_ai_ci	16,0 kio	-
☐ reading_lists	★ Parcourir Structure Rechercher Insérer Vider Supprimer	2	InnoDB	utf8mb4_0900_ai_ci	32,0 kio	-
☐ users	★ Parcourir Structure Rechercher Insérer Vider Supprimer	5	InnoDB	utf8mb4_0900_ai_ci	48,0 kio	-
☐ user_events	★ Parcourir Structure Rechercher Insérer Vider Supprimer	54	InnoDB	utf8mb4_0900_ai_ci	16,0 kio	-
6 tables	Somme	91	InnoDB	utf8mb4_0900_ai_ci	176,0 kio	0 o

Les relations entre ces tables sont matérialisées par des clés étrangères avec suppression en cascade (ON DELETE CASCADE) : reading_lists référence users, list_books référence reading_lists, et comments référence users. Ce qui fait que supprimer une liste de lecture supprime automatiquement les livres qu'elle contenait, et supprimer un compte supprimerait automatiquement ses listes et ses commentaires associés.

```

--
ALTER TABLE `comments`
  ADD CONSTRAINT `comments_ibfk_1` FOREIGN KEY (`user_id`) REFERENCES `users` (`id`) ON DELETE CASCADE;

--
-- Contraintes pour la table `list_books`
--
ALTER TABLE `list_books`
  ADD CONSTRAINT `list_books_ibfk_1` FOREIGN KEY (`list_id`) REFERENCES `reading_lists` (`id`) ON DELETE CASCADE;

--
-- Contraintes pour la table `reading_lists`
--
ALTER TABLE `reading_lists`
  ADD CONSTRAINT `reading_lists_ibfk_1` FOREIGN KEY (`user_id`) REFERENCES `users` (`id`) ON DELETE CASCADE;
COMMIT;

```

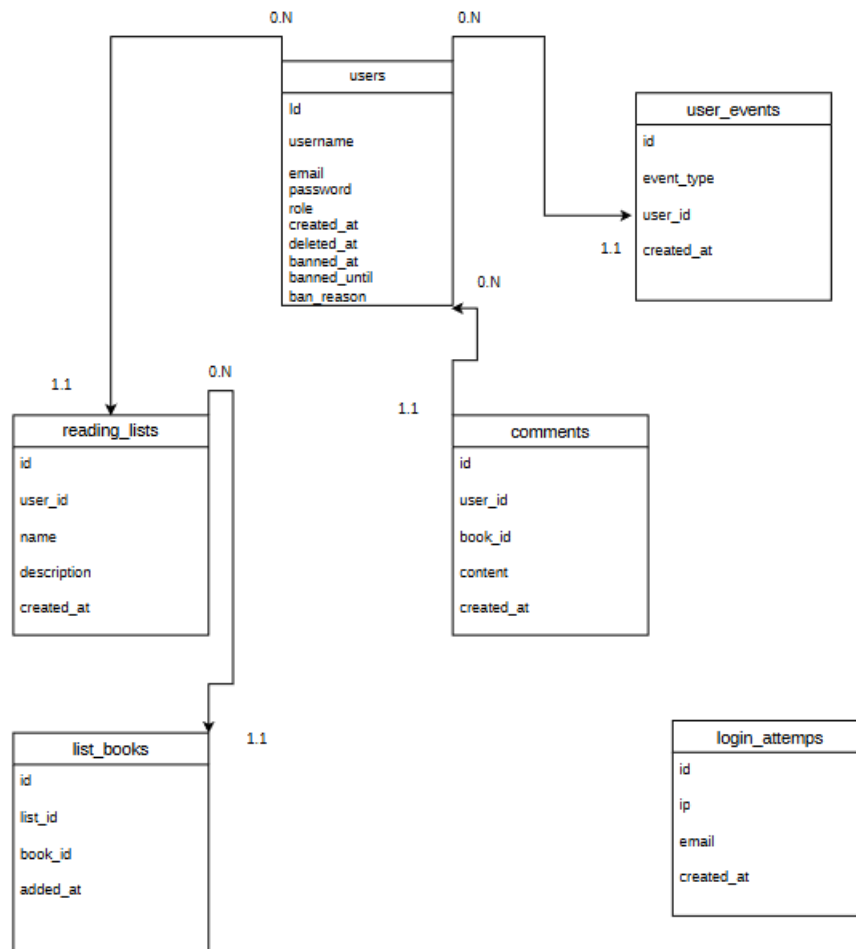
login_attempts et user_events se distinguent des autres tables : malgré leurs colonnes email ou user_id, elles ne portent aucune contrainte de clé étrangère vers users. Pour login_attempts, le choix est dû au fait que la table doit pouvoir enregistrer des tentatives de connexion correspondant à un email erroné ou inexistant, ce qu'une clé étrangère interdirait. Pour user_events, en revanche, rien ne justifie cette absence : la suppression d'un compte (deleteUser) ne supprime jamais la ligne users mais l'anonymise par un UPDATE (email et username remplacés, deleted_at renseigné) .

Au niveau des colonnes, plusieurs contraintes UNIQUE servent à garantir la cohérence des données. Sur users, elles empêchent la création de deux comptes avec le même email ou le même nom d'utilisateur. Sur list_books, la contrainte porte sur la combinaison des deux colonnes list_id et book_id plutôt que sur une seule : elle empêche qu'un même livre soit ajouté deux fois à la même liste, tout en permettant qu'un user ai le même livre dans plusieurs listes différentes.

Le rôle d'un utilisateur et le type d'un événement sont définis par des colonnes ENUM ('user'/'admin', 'register'/'login'/'delete'), ce qui restreint les valeurs possibles directement au niveau de la base, plutôt que de s'appuyer uniquement sur la validation applicative. Les colonnes liées au bannissement (banned_at, banned_until, ban_reason) et à l'anonymisation (deleted_at) sont nullables : leur absence de valeur signifie qu'un compte n'a jamais été banni ou anonymisé.

```
CREATE TABLE users (
  `id` int NOT NULL,
  `username` varchar(20) CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci NOT NULL,
  `email` varchar(100) NOT NULL,
  `password` varchar(255) NOT NULL,
  `role` enum('user','admin') DEFAULT 'user',
  `created_at` datetime DEFAULT CURRENT_TIMESTAMP,
  `deleted_at` datetime DEFAULT NULL,
  `banned_at` datetime DEFAULT NULL,
  `banned_until` datetime DEFAULT NULL,
  `ban_reason` varchar(150) CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci DEFAULT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

1) MCD



Relation : users (0,N) — (1,1) reading_lists

- users (0,N) : un user peut être associé à 0, 1 ou plusieurs reading_lists
- reading_lists (1,1) : une liste est toujours associée à exactement 1 user, jamais 0

Relation : users (0,N) — (1,1) comments

- users (0,N) : un user peut laisser 0, 1 ou plusieurs commentaires.
- comments (1,1) : un commentaire est toujours associé à exactement 1 user, jamais 0

Relation : reading_lists (0,N) — (1,1) list_books

- reading_lists (0,N) : une liste peut contenir 0, 1 ou plusieurs livres.
- list_books (1,1) : une ligne list_books est toujours associée à exactement 1 liste, jamais 0

Relation : users (0,N) — (0,1) user_events

- Côté users (0,N) : un user peut générer 0, 1 ou plusieurs events (un register, puis plusieurs login au fil du temps).
- Côté user_events (1,1) : un event est toujours associé à exactement un user.

login_attempts : aucune relation

- Pas de couple à donner : ip et email sont des colonnes servant à journalisées, sans clé étrangère vers users car la table doit rester indépendante, afin de pouvoir enregistrer des tentatives qui ne correspondent à aucun compte existant.

2)MLD

USERS (id, username, email, password, role, created_at, deleted_at, banned_at, banned_until, ban_reason)

READING_LISTS (id, #user_id, name, description, created_at)

LIST_BOOKS (id, #list_id, book_id, added_at)

COMMENTS (id, #user_id, book_id, content, created_at)

USER_EVENTS (id, event_type, #user_id, created_at)

LOGIN_ATTEMPTS (id, ip, email, created_at)

E) Développer des composants d'accès aux données SQL et NoSQL (cp6)

Le dossier `models` regroupe les requêtes SQL : un fichier par logique fonctionnelle, chacun construit sur `mysql2/promise` avec des requêtes préparées. Cette couche fait l'interface entre les contrôleurs et la base de données, et c'est elle qui porte la responsabilité d'accéder aux données de façon sécurisée et cohérente.

`user-model.js` regroupe tout ce qui concerne la gestion d'un compte : recherche par email ou par identifiant, création, mise à jour du mot de passe, suppression sous forme d'anonymisation, ainsi que le suivi des tentatives de connexion échouées et des commentaires récents d'un utilisateur.

`comment-model.js` couvre le cycle de vie complet d'un commentaire : lecture (jointe au nom de son auteur), création, modification, suppression par son auteur, et suppression par un administrateur.

`list-model.js` gère les listes de lecture et leur contenu : création et suppression d'une liste, vérification de son propriétaire, ajout et retrait d'un livre, et consultation du contenu d'une liste.

`admin-model.js` regroupe les requêtes nécessaires à la modération : listage paginé des utilisateurs, des commentaires et des comptes bannis, ainsi que les actions de bannissement et de débannissement.

1) `admin-model.js`

Les trois fonctions de listage (`getAllUsers`, `getAllComments`, `getBannedUsers`) suivent le même schéma : une requête principale paginée avec `LIMIT/OFFSET`, accompagnée d'une deuxième requête de comptage total, nécessaire pour calculer le nombre de pages côté contrôleur.

```

export async function getAllUsers(page, limit) {
  const offset = (page - 1) * limit;

  const [rows] = await db.query(
    `SELECT id, username, email, role, created_at FROM users
    WHERE deleted_at IS NULL
    AND (banned_at IS NULL OR (banned_until IS NOT NULL AND banned_until <= NOW()))
    ORDER BY created_at DESC LIMIT ? OFFSET ?`,
    [limit, offset]
  );

  const [countRows] = await db.execute(
    `SELECT COUNT(*) as total FROM users
    WHERE deleted_at IS NULL
    AND (banned_at IS NULL OR (banned_until IS NOT NULL AND banned_until <= NOW()))`
  );

  return { data: rows, total: countRows[0].total };
}

```

getAllUsers exclut les comptes actuellement bannis de la liste générale : la condition (banned_at IS NULL OR (banned_until IS NOT NULL AND banned_until <= NOW())) ne retient que les comptes jamais bannis, ou dont le bannissement temporaire est déjà expiré. Les comptes bannis actifs n'apparaissent que dans les tableaux dédiés, via getBannedUsers.

getBannedUsers distingue les bannissements temporaires des bannissements définitifs grâce à typeFilter, un fragment de condition SQL choisi selon le paramètre type avant d'être inséré dans la requête. Ce fragment n'est pas une valeur mais une portion de syntaxe SQL : il ne peut donc pas être transmis comme un paramètre lié, contrairement à LIMIT et OFFSET juste à côté.

banUser et unbanUser servent à faire des mises à jour : le premier renseigne banned_at, banned_until et ban_reason, le second les remet à NULL.

```

export async function banUser(userId, reason, bannedUntil) {
  const [result] = await db.execute(
    `UPDATE users SET banned_at = NOW(), banned_until = ?, ban_reason = ? WHERE id = ? AND deleted_at IS NULL`,
    [bannedUntil ?? null, reason ?? null, userId]
  );
  return result.affectedRows > 0;
}

export async function unbanUser(userId) {
  const [result] = await db.execute(
    `UPDATE users SET banned_at = NULL, banned_until = NULL, ban_reason = NULL WHERE id = ? AND deleted_at IS NULL`,
    [userId]
  );
  return result.affectedRows > 0;
}

```

2)comment-model.js

getCommentsByBooks récupère les commentaires d'un livre en une seule requête via une jointure avec users qui ramène directement le nom de l'auteur du commentaire.

```
export async function getCommentsByBooks(bookId) {
  const [rows] = await db.execute(
    `SELECT c.id, c.content, c.created_at, u.username
    FROM comments c
    JOIN users u ON c.user_id = u.id
    WHERE c.book_id = ?
    ORDER BY c.created_at DESC`,
    [bookId]
  );
  return rows;
}
```

deleteComment et updateComment intègrent la vérification d'autorisation dans la clause WHERE (AND user_id = ?).

deleteCommentAdmin, supprime un commentaire par son seul identifiant, sans condition de propriétaire : la fonction n'est utilisée que sur les routes protégées par le middleware isAdmin, qui réserve son usage aux administrateurs afin qu'ils puissent modérer.

getLastTimeComment sert à appliquer un délai minimum de cinq minutes entre deux commentaires d'un même utilisateur.

```

export async function deleteComment(commentId, userId) {
  const [result] = await db.execute(
    'DELETE FROM comments WHERE id = ? AND user_id = ?',
    [commentId, userId]
  );
  return result.affectedRows > 0;
}

export async function updateComment(commentId, userId, content) {
  const [result] = await db.execute(
    'UPDATE comments SET content = ? WHERE id = ? AND user_id = ?',
    [content, commentId, userId]
  );
  return result.affectedRows > 0;
}

export async function deleteCommentAdmin(commentId) {
  const [result] = await db.execute(
    'DELETE FROM comments WHERE id = ?',
    [commentId]
  );
  return result.affectedRows > 0;
}

export async function getLastTimeComment(userId) {
  const [rows] = await db.execute (
    'SELECT created_at FROM comments WHERE user_id = ? ORDER BY created_at DESC LIMIT 1',
    [userId]
  );
  return rows[0]?.created_at ?? null;
}

```

3)list-model.js

getListsByUser et createList sont des opérations de lecture et création de liste. GetListOwner, sert de brique de base au middleware isListOwner : avant d'autoriser la consultation, l'ajout ou le retrait d'un livre dans une liste (routes /:id/books), ce middleware appelle getListOwner pour vérifier que l'utilisateur connecté est bien le propriétaire, avant même d'exécuter la fonction du contrôleur.


```

export async function getListsByUser(userId) {
  const [rows] = await db.execute(
    'SELECT id, name, description, created_at FROM reading_lists WHERE user_id = ?',
    [userId]
  );
  return rows;
}

export async function createList(userId, name, description) {
  const [result] = await db.execute(
    'INSERT INTO reading_lists (user_id, name, description) VALUES (?, ?, ?)',
    [userId, name, description ?? null]
  );
  return result.insertId;
}

export async function getListOwner(listId) {
  const [rows] = await db.execute(
    'SELECT user_id FROM reading_lists WHERE id = ?',
    [listId]
  );
  return rows[0]?.user_id ?? null;
}

```

La suppression d'une liste entière (deleteList) fait la vérification de propriétaire via la clause WHERE (id = ? AND user_id = ?).

```

export async function deleteList(listId, userId) {
  const [result] = await db.execute(
    'DELETE FROM reading_lists WHERE id = ? AND user_id = ?',
    [listId, userId]
  );
  return result.affectedRows > 0;
}

```

isBookInList vérifie qu'un livre n'est pas déjà présent dans une liste avant son ajout. Cette vérification applicative est doublée par la contrainte UNIQUE sur (list_id, book_id)

```
export async function isBookInList(listId, bookId) {
  const [rows] = await db.execute(
    'SELECT 1 FROM list_books WHERE list_id = ? AND book_id = ? LIMIT 1',
    [listId, bookId]
  );
  return rows.length > 0;
}
```

4)user-model.js

findUserByEmail, findUserById et getUserById filtrent systématiquement avec AND deleted_at IS NULL : un compte anonymisé devient invisible à toute recherche, qu'il s'agisse d'une connexion, d'une vérification de session ou d'une consultation de profil.

```
export async function findUserById(userId) {
  const [rows] = await db.execute(
    'SELECT id, username, email, password, banned_at, banned_until FROM users WHERE id = ? AND deleted_at IS NULL',
    [userId]
  );
  return rows[0] ?? null;
}

export async function getUserById(userId) {
  const [rows] = await db.execute(
    'SELECT id, username, email, created_at FROM users WHERE id = ? AND deleted_at IS NULL',
    [userId]
  );
  return rows[0] ?? null;
}
```

deleteUser réalise l'anonymisation en deux temps : il récupère d'abord l'email du compte, avant qu'il ne soit modifié, pour supprimer les tentatives de connexion associées dans login_attempts. Le compte est ensuite mis à jour avec des valeurs anonymisées (email, nom d'utilisateur, mot de passe), plutôt que d'être supprimé.

```

export async function deleteUser(userId) {
  const [userRows] = await db.execute(
    `SELECT email FROM users WHERE id = ? AND deleted_at IS NULL`,
    [userId]
  );
  if (userRows.length === 0) return false;

  await db.execute(`DELETE FROM login_attempts WHERE email = ?`, [userRows[0].email]);

  const [result] = await db.execute(
    `UPDATE users SET
      deleted_at = NOW(),
      email = CONCAT('deleted_', id, '@deleted.local'),
      username = CONCAT('utilisateur_supprime_', id),
      password = 'DELETED'
    WHERE id = ? AND deleted_at IS NULL`,
    [userId]
  );
  return result.affectedRows > 0;
}

```

getRecentCommentsByUser alimente la page de profil avec les commentaires paginés d'un utilisateur, en suivant le même schéma db.query/db.execute que les fonctions de listage d'admin-model.js.

```

export async function getRecentCommentsByUser(userId, page, limit) {
  const offset = (page - 1) * limit;
  const [rows] = await db.query(
    `SELECT id, content, book_id, created_at FROM comments
    WHERE user_id = ? ORDER BY created_at DESC
    LIMIT ? OFFSET ?`,
    [userId, limit, offset]
  );
  const [countRows] = await db.execute(
    `SELECT COUNT(*) as total FROM comments WHERE user_id = ?`,
    [userId]
  );
  return { data: rows, total: countRows[0].total };
}

```

createUser force le rôle à 'user', sans jamais accepter de valeur transmise pour ce champ. Aucune inscription ne peut donc créer un compte administrateur, même en manipulant les données envoyées au serveur, ce qui fait que rôle n'est attribuable peut être attribué que directement en base.

```
export async function createUser(username, email, hashedPassword) {
  const [result] = await db.execute(
    'INSERT INTO users (username, email, password, role) VALUES (?, ?, ?, ?)',
    [username, email, hashedPassword, 'user']
  );
  return result.insertId;
}
```

logUserEvent, countRecentFailedAttempts et logFailedAttempt alimentent les deux tables de traçabilité. La fenêtre de comptage des tentatives échouées est glissante sur six heures (DATE_SUB(NOW(), INTERVAL 6 HOUR)), ce qui correspond au mécanisme anti brute-force qui est dans auth-controller.js.

```
export async function logUserEvent(event_type, user_id) {
  await db.execute(
    'INSERT INTO user_events (event_type, user_id) VALUES (?, ?)',
    [event_type, user_id]
  );
}
```

F) Développer des composants métier coté serveur (CP7)

Cette partie regroupe les composants métier côté serveur : les contrôleurs (auth, admin, comment, list, user) qui portent la logique de chaque fonctionnalité, les middlewares de sécurité (isAuthenticated, isAdmin, isListOwner, errorHandler) qui protègent les routes ainsi que les fichiers de routes qui composent ces middlewares pour chaque endpoint.

Plusieurs aspects sont couverts : la sécurité de l'authentification (hachage des mots de passe, anti brute-force, gestion du jeton JWT), les règles métier propres à chaque fonctionnalité (limite de livres par liste, délai anti-spam sur les commentaires, impossibilité de s'autobannir), une vérification supplémentaire avant tout changement de mot de passe, et une gestion centralisée des erreurs en bout de chaîne.

Les tests unitaires Vitest couvrent les schémas de validation Zod utilisés dans les contrôleurs ainsi que le middleware isAuthenticated, et un jeu d'essai réalisé avec Postman illustre le comportement de l'inscription face à des données valides et invalides.

1) Dossier routes

Chaque fichier de routes/ déclare les endpoints d'une ressource et la chaîne de middlewares qui les protège, avant que la requête n'atteigne le moindre contrôleur.

auth-routes.js (inscription, connexion, déconnexion) et books-routes.js (recherche de livres) n'ont pas besoin de middleware car ce sont des points d'entrée publics, accessibles avant même d'avoir une session.

```
import { Router } from 'express';
import { register, login, logout } from '../controllers/auth-controller.js';

export const authRouter = Router();

authRouter.post('/register', register);
authRouter.post('/login', login);
authRouter.post('/logout', logout);
```

comments_routes.js la lecture des commentaires reste publique, mais la création, la modification et la suppression passent chacune par isAuthenticated.

```
import { Router } from 'express';
import { isAuthenticated } from '../middlewares/isAuthenticated.js';
import { getComments, createCommentHandler, deleteCommentHandler, updateCommentHandler } from '../controllers/comment-controller.js';

export const commentsRouter = Router();

commentsRouter.get('/:bookId', getComments);
commentsRouter.post('/:bookId', isAuthenticated, createCommentHandler);
commentsRouter.delete('/:id', isAuthenticated, deleteCommentHandler);
commentsRouter.put('/:id', isAuthenticated, updateCommentHandler);
```

lists-routes.js et user-routes.js appliquent isAuthenticated à l'ensemble de leurs routes via .use(), puisque aucune liste ni aucun profil ne doit être consultable sans session. lists-routes.js ajoute en plus isListOwner, mais seulement sur les trois routes /:id/books. La vérification du propriétaire pour la suppression de liste se fait via requête SQL (clause WHERE id = ? AND user_id = ? de deleteList, dans list-model.js).

```

1 import { Router } from 'express';
2 import { isAuthenticated } from '../middlewares/isAuthenticated.js';
3 import { isListOwner } from '../middlewares/isListOwner.js';
4 import { getLists, createListHandler, deleteListHandler, getBooks, addBook, removeBook } from
5
6 export const listsRouters = Router();
7
8 listsRouters.use(isAuthenticated);
9
10 listsRouters.get('/', getLists);
11 listsRouters.post('/', createListHandler);
12 listsRouters.delete('/:id', deleteListHandler);
13 listsRouters.get('/:id/books', isListOwner, getBooks);
14 listsRouters.post('/:id/books', isListOwner, addBook);
15 listsRouters.delete('/:id/books/:bookId', isListOwner, removeBook);
16

```

admin-routes.js a deux middlewares sur toutes ses routes : isAuthenticated puis isAdmin, Ce qui fait qu'une requête sans session échoue avec isAuthenticated et une requête d'un utilisateur connecté mais non administrateur échoue au second.

```

import { Router } from 'express';
import { isAuthenticated } from '../middlewares/isAuthenticated.js';
import { isAdmin } from '../middlewares/isAdmin.js';
import { getUsersHandler, getCommentsHandler, getBannedUsersHandler, deleteCommentAdminHandler, banUserHandler, unbanUserHandler } from

export const adminRouter = Router();

adminRouter.use(isAuthenticated);
adminRouter.use(isAdmin);

adminRouter.get('/users', getUsersHandler);
adminRouter.get('/comments', getCommentsHandler);
adminRouter.delete('/comments/:id', deleteCommentAdminHandler);
adminRouter.delete('/users/:id/ban', unbanUserHandler);
adminRouter.get('/banned-users', getBannedUsersHandler);
adminRouter.patch('/users/:id/ban', banUserHandler);

```

2) Dossier controllers

a) admin-controller.js

Les trois fonctions de listage (getUsersHandler, getCommentsHandler, getBannedUsersHandler) appliquent le même traitement de page : `Math.max(1, Number(req.query.page) || 1)` sert à garantir un nombre entier au moins égal à 1, quelle que soit la valeur reçue dans l'URL, une chaîne non numérique devient NaN, puis retombe sur 1 grâce au `||`.

```

const LIMIT = 15;

export async function getUsersHandler(req, res) {
  const page = Math.max(1, Number(req.query.page) || 1);
  const { data, total } = await getAllUsers(page, LIMIT);

  res.json({
    data,
    page,
    total,
    totalPages: Math.ceil(total / LIMIT),
  });
}

```

getBannedUsersHandler protège contre l'injection SQL sur le paramètre type .

```

export async function getBannedUsersHandler(req, res) {
  const page = Math.max(1, Number(req.query.page) || 1);
  const type = req.query.type === 'permanent' ? 'permanent' : 'temp';
  const { data, total } = await getBannedUsers(page, LIMIT, type);

  res.json({
    data,
    page,
    total,
    totalPages: Math.ceil(total / LIMIT),
  });
}

```

Les trois actions de modération (suppression de commentaire, bannissement, débannissement) commencent toutes par vérifier que l'identifiant reçu est un nombre valide, avant toute autre opération.

banUserHandler ajoute une vérification supplémentaire propre à cette action : un administrateur ne peut pas se bannir lui-même (`userId === req.user.userId`). La durée du bannissement est ensuite calculée à partir du nombre de jours reçu ou laissée à null pour un bannissement permanent .

```

export async function banUserHandler(req, res) {
  const userId = Number(req.params.id);
  if (isNaN(userId)) {
    res.status(400).json({ message: "cet id est invalide " });
    return;
  }

  if (userId === req.user.userId) {
    res.status(403).json({ message: "il n'est pas possible de t'autobannir " });
    return;
  }

  const { reason, duration_days } = req.body;
  const bannedUntil = duration_days
    ? new Date(Date.now() + duration_days * 86400000)
    : null;

  const banned = await banUser(userId, reason, bannedUntil);
  if (!banned) {
    res.status(404).json({ message: "L'utilisateur n'existe pas" });
    return;
  }

  res.status(204).send();
}

```

b)auth-controller.js

register valide les données avec RegisterSchema, puis vérifie si l'email est déjà utilisé avant de créer le compte. Le mot de passe est haché avec bcrypt (coût 12) avant d'être stocké, et l'inscription est journalisée dans user_events.

login sert à vérifier le nombre de tentatives échouées récentes pour l'adresse IP, avant même de chercher l'utilisateur en base : une IP déjà bloquée n'entraîne aucune recherche supplémentaire. Le mot de passe n'est comparé que si un compte correspondant à l'email existe (user ? await bcrypt.compare(...) : false) .

Le statut de bannissement n'est vérifié qu'après validation du mot de passe.

Une fois ces vérifications passées, un jeton JWT est signé avec l'identifiant et le rôle de l'utilisateur, une durée d'expiration lue depuis la variable d'environnement JWT_EXPIRES_IN (30 minutes), puis transmis dans un cookie httpOnly et sameSite=strict.

logout efface ce cookie en reprenant exactement les mêmes options (httpOnly, sameSite) que celles utilisées pour le créer .

c)comment-controller.js

getComments et createCommentHandler valident tous les deux l'identifiant du livre reçu dans l'URL avec BookIdSchema, avant de l'utiliser.

createCommentHandler valide ensuite le contenu du commentaire avec CommentSchema, puis vérifie le délai écoulé depuis le dernier commentaire de cet utilisateur grâce à getLastTimeComment. Si moins de cinq minutes se sont écoulées, la requête est refusée et un message précisant le temps restant à attendre, calculé et formaté en minutes et secondes.

```
export async function createCommentHandler(req, res) {
  const parsedBookId = BookIdSchema.safeParse(req.params.bookId);
  if (!parsedBookId.success) {
    res.status(400).json({ message: "L'identifiant du livre est invalide" });
    return;
  }

  const parsed = CommentSchema.safeParse(req.body);
  if (!parsed.success) {
    res.status(400).json({ errors: z.flattenError(parsed.error).fieldErrors });
    return;
  }

  const lastComment = await getLastTimeComment(req.user.userId);
  if (lastComment) {
    const diff = Date.now() - new Date(lastComment).getTime();
    if (diff < 300000) {
      const totalSeconds = Math.ceil((300000 - diff) / 1000);
      const minutes = Math.floor(totalSeconds / 60);
      const seconds = totalSeconds % 60;
      const remaining = minutes > 0
        ? `${minutes} minute${minutes > 1 ? 's' : ''} et ${seconds} seconde${seconds > 1 ? 's' : ''}`
        : `${seconds} seconde${seconds > 1 ? 's' : ''}`;
      res.status(429).json({
        message: `Vous devez attendre ${remaining} avant de poster un nouveau commentaire.`,
      });
      return;
    }
  }

  await createComment(req.user.userId, parsedBookId.data, parsed.data.content);

  res.status(201).json({ message: 'Le commentaire a été posté' });
}
```

deleteCommentHandler et updateCommentHandler commencent par vérifier que l'identifiant du commentaire est un nombre valide, puis délèguent la vérification du propriétaire aux modèles (deleteComment et updateComment) et si la suppression ou la modification ne correspond à aucune ligne en base, la fonction renvoie false et le contrôleur répond 404.

d)list-model.js

getLists ne reçoit aucun identifiant de liste car elle renvoie directement les listes de l'utilisateur connecté (getListsByUser(req.user.userId)). Il n'est pas possible de consulter les listes de quelqu'un d'autre par ce chemin, puisque la requête part toujours de son propre identifiant.

createListHandler valide les données avec CreateListSchema avant de créer la liste qui est rattachée à l'utilisateur .

deleteListHandler vérifie que l'identifiant reçu est un nombre, puis deleteList gère la vérification de propriétaire..

getBooks récupère les livres d'une liste, addBook en ajoute un, removeBook en retire un. La vérification de l'utilisateur est déléguée au middleware isListOwner

addBook applique deux règles avant d'ajouter un livre , il n'est pas permis d'avoir un doublon dans une liste (isBookInList) et ne permet pas qu'il y ait plus de dix livres par liste (countBooksInList)

```
export async function addBook(req, res) {
  const listId = Number(req.params.id);
  const { bookId } = req.body;

  if (!bookId) {
    res.status(400).json({ message: 'Les données sont invalides' });
    return;
  }

  const alreadyInList = await isBookInList(listId, bookId);
  if (alreadyInList) {
    res.status(409).json({ message: 'Ce livre est déjà dans la liste' });
    return;
  }

  const total = await countBooksInList(listId);
  if (total >= maxBooks) {
    res.status(409).json({ message: `Une liste ne peut pas contenir plus de ${maxBooks} livres` });
    return;
  }

  await addBooksToList(listId, bookId);
  res.status(201).json({ message: 'Le livre a été ajouté' });
}
```

e)user-controllers.js

getProfile renvoie les informations du compte connecté et ses commentaires récents paginés, en un seul appel combinant deux sources de données.

changePassword sert vérifier que les deux mots de passe sont fournis, puis confirme l'ancien avec `bcrypt.compare` avant d'accepter le nouveau, haché à son tour.

```
export async function changePassword(req, res) {
  const { currentPassword, newPassword } = req.body;

  if (!currentPassword || !newPassword) {
    res.status(400).json({ message: 'Les deux mots de passe sont requis' });
    return;
  }

  const user = await findUserById(req.user.userId);
  if (!user) {
    res.status(404).json({ message: 'Utilisateur introuvable' });
    return;
  }

  const valid = await bcrypt.compare(currentPassword, user.password);
  if (!valid) {
    res.status(401).json({ message: "ce n'est pas votre mot de passe actuel "});
    return;
  }

  const hashed = await bcrypt.hash(newPassword, 12);
  await updatePassword(req.user.userId, hashed);

  res.json({ message: "le mot de passe a bien été modifié " });
}
```

deleteAccount déclenche l'anonymisation du compte (`deleteUser`), puis efface le cookie de session.

```
export async function deleteAccount(req, res) {
  const deleted = await deleteUser(req.user.userId);
  if (!deleted) {
    res.status(404).json({ message: 'Utilisateur introuvable' });
    return;
  }

  res.clearCookie('token', { httpOnly: true, secure: false, sameSite: 'strict' });
  res.json({ message: "le compte a bien été supprimé " });
}
```

3) Dossier Middleware

a) errorHandler.js

Il sert à intercepter toute erreur qui n'a pas été traitée explicitement par un contrôleur. Il enregistre l'erreur dans les logs serveur, tout en envoyant un message générique au client, jamais le détail technique de l'erreur par mesure de précaution.

```
export function errorHandler(err, req, res, next) {
  console.error(`[${new Date().toISOString()}] ${req.method} ${req.path}`, err);
  res.status(500).json({ message: 'Une erreur interne est survenue.' });
}
```

b) isAdmin.js

Il sert à vérifier le rôle présent dans req.user (qui est géré par isAuthenticated). Si l'utilisateur n'est pas admin, la requête est rejetée.

```
export function isAdmin(req, res, next) {
  if (req.user?.role !== 'admin') {
    res.status(403).json({ message: 'Cet accès ne vous est pas permis' });
    return;
  }
  next();
}
```

c) isAuthenticated.js

isAuthenticated sert à vérifier que le jeton JWT est bien dans le cookie de la requête. Sans jeton, ou si la vérification échoue (signature invalide, jeton expiré), la requête est rejetée. Si le jeton est valide, le middleware vérifie aussi que le compte n'est pas banni, renouvelle le jeton pour encore trente minutes, puis attache l'utilisateur à req.user avant de laisser passer la requête vers le contrôleur.

d)isListOwner.js

isListOwner sert à vérifier que l'utilisateur est bien le propriétaire possède de la liste ciblée par l'identifiant de l'URL, via getListOwner. S'il n'en est pas le propriétaire, la requête est rejetée.

```
import { getListOwner } from '../models/list-model.js';

export async function isListOwner(req, res, next) {
  const listId = Number(req.params.id);
  if (isNaN(listId)) {
    res.status(400).json({ message: "l'id est invalide" });
    return;
  }

  const ownerId = await getListOwner(listId);
  if (ownerId !== req.user.userId) {
    res.status(404).json({ message: 'La liste est introuvable' });
    return;
  }

  next();
}
```

4)les tests vitest et postman

Les tests Vitest couvrent les schémas de validation Zod utilisés par auth-controller.js (RegisterSchema, LoginSchema) ainsi que le middleware isAuthenticated.

```

fauco@Joss MINGW64 ~/Desktop/projet perso/bibliotech (main)
$ npm test

> bibliosauv@1.0.0 test
> vitest

DEV v4.1.5 C:/Users/fauco/Desktop/projet perso/bibliotech

✓ src/shared/schemas/comment-schema.test.js (4 tests) 5ms
✓ src/shared/schemas/list-schema.test.js (6 tests) 6ms
✓ src/shared/schemas/user-schema.test.js (13 tests) 9ms
stdout | src/backend/middlewares/isAuthenticated.test.js
◇ injected env (9) from .env // tip: 🔒 encrypted .env [www.dotenvx.com]

✓ src/backend/middlewares/isAuthenticated.test.js (5 tests) 6ms

Test Files 4 passed (4)
  Tests 28 passed (28)
Start at 14:19:46
Duration 558ms (transform 168ms, setup 0ms, import 674ms, tests 26ms, environment 0ms)

PASS Waiting for file changes...
press h to show help, press q to quit

```

Les tests via Postman servent à compléter les tests automatisés par une vérification manuelle.

POST http://localhost:3023/api/auth/register

Send

Docs Params Authorization Headers 8 Body Scripts Tests Settings

Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON

Schema Beautify

```
1 {  
2   "username": "tes",  
3   "email": "test@tes.fr",  
4   "password": "Passpass!9213"  
5 }
```

Body Cookies Headers 9 Test Results

201 Created • 399 ms • 350 B • Save Response

{ } JSON Preview Visualize

```
1 {  
2   "message": " le Compte a bien été crée"  
3 }
```

III) Axe d'amélioration

La documentation de l'API via Swagger/OpenAPI n'est pas mise en place. Cela permettrait de fournir une documentation efficace.

Il n'y a pas de mise en cache des données provenant d'Open Library . Cela permettrait de réduire la dépendance de l'application , et de limiter le nombre de requêtes envoyées pour des données qui changent rarement, comme les livres tendance du jour et de reduire les erreurs du à la l'api.

Les tests automatisés sont incomplets comme l'indique test:coverage . Seuls les schémas de validation Zod et le middleware d'authentification .

```
fauco@Joss MINGW64 ~/Desktop/projet perso/bibliotech (main)
● $ npm run test:coverage

> bibliosauv@1.0.0 test:coverage
> vitest run --coverage

RUN v4.1.5 C:/Users/fauco/Desktop/projet perso/bibliotech
Coverage enabled with v8

✓ src/shared/schemas/list-schema.test.js (6 tests) 6ms
✓ src/shared/schemas/comment-schema.test.js (4 tests) 6ms
✓ src/shared/schemas/user-schema.test.js (13 tests) 10ms
stdout | src/backend/middlewares/isAuthenticated.test.js
◇ injected env (9) from .env // tip: % multiple files { path: ['.env.local', '.env'] }

✓ src/backend/middlewares/isAuthenticated.test.js (5 tests) 6ms

Test Files 4 passed (4)
Tests 28 passed (28)
Start at 15:50:46
Duration 394ms (transform 140ms, setup 0ms, import 485ms, tests 28ms, environment 0ms)

% Coverage report from v8
```

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	57.14	52.63	23.07	57.4	
backend	100	100	100	100	
db.js	100	100	100	100	
backend/middlewares	100	90.9	100	100	
isAuthenticated.js	100	90.9	100	100	22
backend/models	0	0	0	0	
user-model.js	0	0	0	0	4-100
shared/schemas	100	100	100	100	
comment-schema.js	100	100	100	100	

🕒 Josselin (1 hour ago)

